

Fiori Elements 開發課程 7：擴充 SAP 標準 App (CDS View Extension，後端層)

⚠️ 環境：這一課換到 On-Premise 系統 (S4H / Client 130，走 sap-adt MCP)，不是 fe01 ~ fe06 / fe08 ~ fe10 用的 BTP ABAP Environment Trial！——原因是這一課要重現團隊內部 2022 年舊 PPT 教材 (EXTEND VIEW + SCFD_REGISTRY)，這份教材示範的是傳統 on-premise 開發模式；下一課 fe08 會切回 BTP Trial (ZRAPCLOUD 套件)。操作方式 (Eclipse ADT 而非 VS Code、SAP GUI SCFD_REGISTRY 而非 BTP Released Objects) 跟前後課次都不一樣，看到本課節奏突然不同不是筆誤。

Lecture

這一課要解決的問題，跟 fe06 的分工

fe06 學過用 **Adaptation Project** 擴充標準 App——那是前端 / UI 層的擴充，不動後端資料模型，新加的欄位如果要顯示真實資料，得自己用 Fragment / Binding 想辦法。這一課要學的是後端 / 資料模型層的擴充：用 **EXTEND VIEW** 加一個真正有資料庫資料支撐的新欄位，再用 **Metadata Extension** 幫它掛上 UI Annotation——這正是團隊裡一份 2022 年的內部教材 (**Fiori 技術分享.pptx**，以 Track Sales Orders / F2577 新增「業務員」欄位為例) 示範的技術。這一課的目標：查證這份 PPT 的做法是不是過時了，如果沒有，在我們自己的系統上重現一次，端對端驗證到畫面上真的看得到新欄位。

官方查詢入口：SCFD_REGISTRY (SAP GUI) + Eclipse **Ctrl+Shift+A**

PPT 給的方法，查證後證實完全是現行有效、正規的官方做法，不需要像本課程其他地方那樣繞 ADT REST API workaround：

第一步：SAP GUI T-code SCFD_REGISTRY (Extensibility Registry)——切到 **Extensible CDS View** 頁籤，用畫面上的搜尋功能 (放大鏡 / Find) 輸入關鍵字 (可以是 App 名稱片段或 CDS View 名稱片段)，會列出所有已標記為可擴充的 **CDS View**，包含 View Name / Description / View Type。這一步的價值不只是「找到 View 名稱」，更重要的是它天生只列「已釋出可擴充」的物件——查到就等於同時確認了可擴充性，跟 fe06 查證 BTP Trial **Released Objects** 分類、跟這一課後面查 **@Metadata.allowExtensions** 是同一種「先確認開放權限，再動手」的精神。

第二步：Eclipse Ctrl+Shift+A (Open ABAP Development Object)——把第一步找到的 CDS View 名稱貼進搜尋框，會一次列出這個名稱底下所有相關物件 (Data Definition / Metadata Extension / Access Control.....)，不用像本課程在 BTP Cloud 環境常常要用的 curl **quickSearch** workaround，因為這台系統走的是傳統 on-premise 開發模式，Eclipse 原生搜尋功能就夠用。

這兩步合起來，就是這一課「查 OData Service / UI Annotation」這個目標最直接的官方答案——不用像 fe06 那樣要嘗試連 BTP Trial、查 **Released Objects**、或用 curl 打 ADT quickSearch API，這台系統上有 SAP GUI 就直接用 **SCFD_REGISTRY**。

已用截圖實測確認：在 **SCFD_REGISTRY** 的 **Extensible CDS View** 頁籤搜尋 **Track sales**，準確篩出 **C_SLSDOCFLFLLMNTANALYZER** (Description 欄位就寫著「Track Sales Orders」)，而且 **View Type Description** 欄位標示「**Retrieval via Association to Extension Include View**」——這句官方登記的說明，精準描述了這個 View 的擴充機制：透過新增 **Association** 接到一個「**Extension Include View**」來取得擴充資

料，跟這一課後面實作的 `association [0..1] to I_SDDocumentCompletePartners as SoldToPartyAddressInfo3 ...` 做法完全對應，等於 `SCFD_REGISTRY` 在查詢階段就已經預告了應該用什麼手法擴充。

⚠️ ⚠️ 真實踩坑記錄：沒有先查證，選錯了目標 CDS View

這一課的第一次嘗試沒有先用 `SCFD_REGISTRY` 查證，而是憑 App 名稱「Track Sales Orders」直覺猜測底層 CDS View 叫 `C_SalesOrderTP`（畢竟名字裡有 SalesOrder）。這個猜測完全錯誤：

1. 用 `curl quickSearch` 找到 `C_SALESORDERTP`，讀原始碼確認它有 `@Metadata.allowExtensions: true`，語法是舊式 `define view ( V1, obsolete )`——一切看起來都合理，於是照著建了 `ZC_SALESORDER_VE01`（CDS View Extension）+ `ZC_SALESORDER_VE01_MDE`（Metadata Extension），兩個都一次就啟用成功，語法完全正確。
2. 到 Fiori Launchpad 打開真正的「Track Sales Orders」App 測試，新欄位完全沒有出現——搜尋篩選清單找不到、`Adapt Filters` 也找不到。
3. 排查發現這個 App 背後的 OData Service 是 `ZSD_SLS_SALESORDER_MANAGE_SRV`，Model 名稱 `ZSD_SLS_SALESORDER_MANAGE_MDL`——`_MDL / _SRV` 字尾是傳統 **SEGW** 專案的命名慣例，代表這個服務的欄位結構是設計時期存好的快照，不會因為擴充了 CDS View 就自動反映，需要另外進 **SEGW** 手動同步（這是這一課意外學到的額外知識：同一個系統上，`_CDS` 字尾的服務是 **SADL** 自動曝露、會即時反映 **CDS** 變化；`_MDL / _SRV` 字尾是傳統 **SEGW**，不會）。
4. 回頭用 `SCFD_REGISTRY` 正規查證，才發現：`C_SalesOrderTP` 根本不是 **Track Sales Orders** 這個 App 的底層 **View**。真正的目標是 `C_SlsDocFllmmtAnalyzer`（Sales Document Fulfillment Analyzer）——這個判斷完全可以從它自己 Metadata Extension 裡的 `@EndUserText.label: 'Track Sales Orders'` 直接確認。

教訓：這一課一開始跳過「查證目標物件」這一步、直接憑名稱猜測，繞了一大圈才發現選錯物件——如果一開始就用 `SCFD_REGISTRY` 查，根本不會走這段冤枉路。這是這門課「查證優先」原則最具體的一次代價示範：查證步驟看起來多花時間，但比起選錯物件後一路做到底才發現要重來，其實省時間非常多。

（第一次嘗試建立的 `ZC_SALESORDER_VE01 / ZC_SALESORDER_VE01_MDE` 保留不刪，當作「選錯目標」的反面教材，見「物件清單」。）

實作：`EXTEND VIEW ( V1` 舊式語法，符合 `C_SlsDocFllmmtAnalyzer` 本身的語法世代)

用 `SCFD_REGISTRY` + Eclipse 確認正確目標後，讀 `C_SlsDocFllmmtAnalyzer` 原始碼確認：
`@Metadata.allowExtensions: true`（可擴充）、`define view ( V1` 舊式語法，因此要用 `EXTEND VIEW` 不是 `EXTEND VIEW ENTITY`）。

CDS View Extension（`ZC_SOFANALYZER_VE01`，套件 `$TMP`）：

```
@AbapCatalog.sqlViewAppendName: 'ZSOFANLZR_APP1'
@EndUserText.label: 'FE07 test - Sales Rep ext v2'
extend view C_SlsDocFllmmtAnalyzer with ZC_SOFANALYZER_VE01
  association [0..1] to I_SDDocumentCompletePartners as SoldToPartyAddressInfo3
    on SalesDocument.SalesDocument = SoldToPartyAddressInfo3.SDDocument
    and SoldToPartyAddressInfo3.SDDocumentItem = '000000'
    and SoldToPartyAddressInfo3.PartnerFunction = 'VE'
{
```

```
@Search.defaultSearchElement: true
@Search.fuzzinessThreshold: 0.8
SoldToPartyAddressInfo3.Personnel
};
```

兩個查證重點：

1. **@AbapCatalog.sqlViewAppendName** 這個 **DDIC Append View** 名稱有長度限制——實測 'ZSOFANALYZER_APP1' (17 碼) 啟用時報錯「Select a shorter name」，縮短成 'ZSOFANLZR_APP1' (14 碼) 就過關，跟這個專案一路記錄過的「DDIC 物件名稱有嚴格長度限制，遇到報錯就縮短重試」是同一套經驗。
2. 關聯的 **ON** 條件要參照原始 **View** 自己的 **FROM** 別名 (**SalesDocument.SalesDocument**)，不是 **\$projection.欄位**——這是因為 **C_SlsDocFlfllmntAnalyzer** 自己的 **select from I_SalesDocument as SalesDocument** 這個別名叫 **SalesDocument**，官方文件說明「可以存取被擴充 View 所有資料來源的欄位」，指的就是這個原始 FROM 別名，不是透過 **\$projection** 間接參照 (**\$projection** 在這裡也不是不能用，但這次照抄 PPT 原本的寫法用 **SalesDocument.SalesDocument**，一次就對)。

實作：Metadata Extension

Metadata Extension (**ZC_SOFANALYZER_VE01_MDE**，套件 **\$TMP**)：

```
@Metadata.layer: #CUSTOMER
annotate view C_SlsDocFlfllmntAnalyzer with
{
  @UI: {
    lineItem: [{ position: 45 }],
    selectionField: [{ position: 25 }],
    fieldGroup: [{ qualifier: 'NLPQuickView', position: 135 }]
  }
  Personnel;
}
```

annotate view 的目標寫的是原始標準 **View** 的名稱 (**C_SlsDocFlfllmntAnalyzer**)，不是我們建的 CDS View Extension 名稱——這是因為 Metadata Extension 的職責是「幫某個 View 的欄位加 UI 標記」，而 **Personnel** 這個欄位 (雖然是透過擴充加進去的) 最終確實屬於 **C_SlsDocFlfllmntAnalyzer** 這個 View 的欄位集合。兩個物件都用 **sap_set_source** 寫入後遇到熟悉的殘留鎖 403 (**.claude/rules/sap-adt-mcp.md** 第 5 節記錄過的常態模式)，照標準流程 **sap_lock**→**sap_unlock**→手動 **curl** 啟用即可排除，都是一次就啟用成功。

多個 Metadata Extension 指向同一個 View：不用改既有的，各自獨立建、框架自動合併

C_SlsDocFlfllmntAnalyzer 這種標準 View，SAP 自己通常已經有一個 (或多個) 標準 Metadata Extension 負責既有欄位 (Sales document / Sold-to Party 這些) 的 **@UI** 標記。我們的 **ZC_SOFANALYZER_VE01_MDE** 不是去改那個既有物件 (跟標準 CDS View 一樣是唯讀的，改不了)，而是另外建一個完全獨立的新物件，兩者剛好都用 **annotate view** 指向同一個 View 名稱——CDS 框架允許同一個 View 被多個 Metadata Extension 同時

標註，啟用 / 執行期會自動把所有指向它的 Metadata Extension 內容合併起來，這也是「Enhancement 疊加而非 Modification」這條原則在 Metadata Extension 這個物件類型上的具體體現。

✅ 已用 Eclipse ADT 畫面實際確認：在 Eclipse 開啟 `C_SLSDOCFLFLLMNTANALYZER` 這個 CDS View 的原始碼，編輯器左側裝訂線會出現一個 **enhance icon**，游標停在上面會彈出一個 tooltip，完整列出這個 View 目前被哪些擴充物件疊加：

- **Extended with** : `ZC_SOFANALYZER_VE01` (我們建的 CDS View Extension · DDLS)
- **Metadata extended with** : 同時列出兩個物件——`C_SLSDOCFLFLLMNTANALYZER` (SAP 自己的標準 Metadata Extension，物件名稱跟被擴充的 **View** 完全同名，這正是官方文件講的 Metadata Extension 命名慣例——SAP 標準內容的 Metadata Extension 習慣跟它擴充的 CDS Entity 同名) 與 `ZC_SOFANALYZER_VE01_MDE` (我們建的)

這張截圖是這一課「多個獨立 Metadata Extension 可以同時指向同一個 View、框架自動合併」這個說法**最直接的證據**——不是靠讀原始碼比對推論出來的，是 Eclipse 這個 tooltip 直接把兩個 MDE 並列給你。也連帶印證了上面「重要澄清」那段的結論：**Extended with** 只列了我們自己的 `ZC_SOFANALYZER_VE01`，代表 Eclipse 認定的「疊加關係」跟「直接修改」是兩種不同性質、會分開顯示的東西。**這個 enhance icon 是比 curl 打 ADT API 查證更快的方法**，之後想確認「某個標準 View 到底被哪些自訂物件擴充了」，優先看這個 tooltip，不用另外查 TADIR 或跑 quickSearch。

多個 Metadata Extension 合併時，誰的標記算數，由 `@Metadata.layer` 決定優先順序 (查證官方文件 `ABENCDS_META_DATA_EXTENSION_EVAL / Propagation of Annotations` 確認)：

```
#CUSTOMER    ← 最高
#PARTNER
#INDUSTRY
#LOCALIZATION
#CORE        ← 最低 ( SAP 標準內容通常屬於這層 )
```

- 系統從最高層開始找同一個欄位、同一個 **annotation**，一找到就用，不會再往更低層找——高層的值直接覆蓋低層，不是內容合併，是「贏者全拿」。`ZC_SOFANALYZER_VE01_MDE` 用的是 `@Metadata.layer: #CUSTOMER` ([fe07_cds_view_extension.md:67](#))，是最高優先層，如果 SAP 標準那個是 `#CORE`，我們天生就贏。
- 同一層裡如果有好幾個 Metadata Extension，系統用「找到的第一個」，順序官方文件形容為「不保證固定但穩定」(**undefined but stable**)——不要設計成依賴同層多個 MDE 互相覆蓋來解決衝突。
- 不管哪一層，Metadata Extension 裡的 **annotation**，優先權永遠高於 CDS View 自己原始碼裡直接寫的 **annotation** (例如標準 View 定義裡欄位前面直接掛的 `@UI.xxx`)——這也是為什麼「用獨立 Metadata Extension 疊加標記」這條路一定能蓋掉 / 補上想要的效果，不會被原始定義擋住。
- **fe07** 這個案例其實完全不會踩到 **Priority** 判斷：Priority 只有在「同一個欄位、同一個 **annotation**，被不只一個 Metadata Extension 標註、且值不一樣」時才需要仲裁。`ZC_SOFANALYZER_VE01_MDE` 只標註我們自己新增的 `Personnel` 欄位——SAP 標準的 Metadata Extension 根本不知道有這個欄位存在，不可能對它下任何 annotation，兩者之間沒有真正的衝突，`#CUSTOMER` 這個宣告更多是「養成習慣、確保萬一將來真的衝突時我們會贏」，不是這次驗證有實際用到覆蓋效果。
- 如果要實際檢查某個 CDS View 的 annotation 最終是從哪個物件、哪一層合併來的，Eclipse ADT 有專用工具：**Active Annotations View** (列出目前生效的 annotation 跟它的來源) 與 **Annotation Propagation View** (同時顯示 active / inactive 的 annotation 值跟各自的來源物件，用顏色區分)。

這三個 @UI 標記各自控制畫面上哪個地方

lineItem / selectionField / fieldGroup 同時標在 Personnel 上，各自獨立控制不同位置，彼此不互斥：

- **@UI.selectionField**——決定這個欄位要不要直接顯示在篩選列 (Filter Bar) 上。查證官方文件 (Configuring Filter Bars) 確認：預設只有標了這個 annotation 的欄位 (加上必填篩選欄位) 才會直接出現在篩選列，沒標的欄位只會待在使用者手動點開的 Adapt Filters 清單裡，要自己勾選才會顯示。這正好解釋了下面「端對端驗證」那段記錄的現象——Personnel 在 Adapt Filters 搜尋清單裡找不到，不是失敗，是因為 @UI.selectionField 已經讓它直接出現在篩選列上，不需要 (也不會出現在) 待加入清單裡。
- **@UI.lineItem**——決定這個欄位要不要顯示成 List Report 表格的一欄，position 決定欄位排序 (fe03 / fe09 / fe12 前面已經用過，這一課第一次正式補上說明)。
- **@UI.fieldGroup**——把多個欄位歸類成同一組、以群組方式顯示 (通常是表單一個接一個排列)，跟 lineItem 是平行機制，只是用途從「表格欄」換成「群組顯示」。qualifier 是這個群組的識別名稱——關鍵在於：不同物件、不同 Metadata Extension 只要用同一個 qualifier 字串，就會被系統歸進同一組 (呼應前面「多個 Metadata Extension 可以同時指向同一個 View、框架自動合併」的原理，只是這裡合併的粒度細到「同一個群組」)。這裡用的 qualifier: 'NLPQuickView' 很可能不是我們自己發明的名字，是 C_SlsDocFlfllmntAnalyzer 標準內容裡本來就存在的一個群組 (推測是點擊 Smart Link / 搜尋結果彈出的 Quick View 卡片要顯示的欄位清單)，把 Personnel 標上同一個 qualifier，等於讓它也加入那個既有卡片的顯示內容，而不是我們自己另開一個新群組。⚠️ 這一點沒有反查系統驗證過 (NLPQuickView 具體對應畫面上哪個彈出卡片)，只是根據命名慣例與官方 Metadata Extension 合併機制的合理推測，之後若要精確確認，需要讀 C_SlsDocFlfllmntAnalyzer 既有標準 Metadata Extension 裡是否也用了同一個 qualifier。

⚠️ ⚠️ 重要澄清：PPT 的用詞容易誤會成「直接改標準物件」，但實際不是

PPT 的操作說明寫著「開啟 Track Sales Orders 的 CDS View」「加入這一段程式碼」「開啟 Metadata Extensions」——這些用詞很容易讓人以為是直接編輯標準物件本身。這一課特地做了驗證：在我們的擴充物件啟用生效之後，重新讀一次 C_SlsDocFlfllmntAnalyzer 的原始碼，內容跟最一開始一模一樣，一個字都沒變。

真相：@Metadata.allowExtensions: true 這個旗標存在的唯一目的，就是開放「建立一個獨立物件來擴充」這條路——SAP 標準物件本身在 Eclipse 裡通常是唯讀的 (沒有 Access Key / SSCR 註冊登記，存不了檔)。PPT 描述的「開啟 CDS View」，實際上是在 Eclipse 對著標準 View 觸發「建立擴充」的精靈流程，表面上有「在這個 View 的上下文裡操作」的錯覺，但背後建立的是全新、獨立命名的物件 (這一課是 ZC_SOFANALYZER_VE01)，標準物件的原始碼從頭到尾沒有被寫入過。這正好呼應這個專案 CLAUDE.md 第一條硬性規則——「不可修改 SAP 標準物件，只能透過 Enhancement / BAdI / User-Exit」，EXTEND VIEW 與 Metadata Extension 正是 CDS 層對應的「Enhancement」機制。

✅ 端對端驗證成功

在 Fiori Launchpad 重新整理 Track Sales Orders (T-code /UI2/FLP 啟動)，點 Adapt Filters 搜尋 Personnel / Per 一開始都搜不到——原因不是失敗，是這個欄位已經自動顯示在篩選列上，不會出現在「可加入」清單裡：比對畫面篩選列，多了一個全新的 Personnel Number: 欄位 (原本只有 Sales document / Sold-To Party / Customer Reference / Requested Deliv.Date / Overall Status / Document Date 六個)。

「Personnel Number」這個顯示文字不是我們自己下的，是繼承自底層 I_SDDocumentCompletePartners-Personnel 欄位原本的標籤——這代表 selectionField Annotation 生效到讓欄位自動顯示在篩選列上，

C_SlsDocFlfllmntAnalyzer 這個 View 走的是 SADL 自動曝露機制 (不是像第一次選錯目標時踩到的傳統 SEGW) , 所以整條鏈路完全即時反映 , 不需要額外的 Metadata 快取刷新。

學習目標

- 知道 **EXTEND VIEW / EXTEND VIEW ENTITY** 是官方現行、沒有過時的機制 , 只是要依照目標 View 的語法世代 (V1 define view 用 **EXTEND VIEW** , V2 define view entity 用 **EXTEND VIEW ENTITY**) 挑對語法
- 能講出查證「哪個 CDS View 支撐哪個標準 Fiori App」的正規官方流程 : SAP GUI **SCFD_REGISTRY** (Extensibility Registry → Extensible CDS View 頁籤) + Eclipse **Ctrl+Shift+A**
- 知道跳過查證、憑名稱猜測目標物件的真實代價——這一課完整記錄了選錯物件 (**C_SalesOrderTP** vs 正確的 **C_SlsDocFlfllmntAnalyzer**) 繞了一大圈才發現的過程
- 能寫出 **EXTEND VIEW ... WITH ...** 的語法 , 知道新關聯的 **ON** 條件要參照原始 View 的 FROM 別名
- 能寫出 **annotate view <原始 View 名稱> with { @UI:{...} 欄位; }** 幫擴充出來的欄位掛 UI Annotation
- 知道 **@AbapCatalog.sqlViewAppendName** 的 DDIC Append View 名稱有長度限制 , 遇到報錯要縮短重試
- 理解「**PPT** 用詞聽起來像改標準物件」是誤會 , 實際上永遠是建立獨立的擴充物件——能用「讀回標準物件原始碼確認沒有變化」這個方法自己驗證
- 知道同一個系統上 , 標準 App 背後的 OData Service 可能是不同世代技術 (**_CDS** 字尾 SADL 自動曝露 vs **_MDL / _SRV** 字尾傳統 SEGW 設計時期快照) , 這會決定 CDS 擴充能不能即時反映到畫面上

物件清單

這一課在 on-premise 系統 (**S4H** , client **130**) 的 **\$TMP** 套件建立驗證物件 , 不需要傳輸請求 :

物件	型別	說明
ZC_SALESORDER_VE01	DDLX (CDS View Extension)	⚠ 第一次嘗試 , 目標選錯 (C_SalesOrderTP) , 保留當反面教材
ZC_SALESORDER_VE01_MDE	DDLX (Metadata Extension)	⚠ 同上 , 配對的錯誤目標 Metadata Extension
ZC_SOFANALYZER_VE01	DDLX (CDS View Extension)	✅ 正確版本 , 擴充 C_SlsDocFlfllmntAnalyzer , 加 Personnel 欄位
ZC_SOFANALYZER_VE01_MDE	DDLX (Metadata Extension)	✅ 正確版本 , 幫 Personnel 掛 @UI Annotation

沒有修改任何標準物件 (**C_SalesOrderTP** / **C_SlsDocFlfllmntAnalyzer** 的原始碼在整個過程中都沒有變化過 , 已用 ADT 讀回原始碼驗證) 。

動手練習

輪到你了 :

1. 用 `SCFD_REGISTRY` 查一個你自己熟悉的其他標準 Fiori App (例如某個 Manage/Track/Display 開頭的 App) , 找出它底層的 CDS View , 確認它有沒有 `@Metadata.allowExtensions: true`
2. 想一想 : 如果 `Adapt Filters` 搜尋新欄位找不到 , 你會先假設是「沒生效」還是「已經自動顯示 , 不需要再加」 ? 這一課教了一個排查方法 (比對篩選列前後有沒有多欄位) , 還有沒有其他方法可以更快確認 (提示 : 想一想 `More Filters (27)` 這種數字提示 , 變成 28 了嗎 ?)
3. 試著查一下你自己找到的那個標準 App 背後的 OData Service , 用 `/n/IWFND/MAINT_SERVICE` 確認它的 Model 名稱字尾是 `_CDS` 還是 `_MDL / _SRV` , 猜猜看如果你也想擴充它 , 新欄位會不會即時反映到畫面上

驗證方式

Eclipse ADT (`sap-adt MCP`) + SAP GUI + Fiori Launchpad 端對端驗證 , 全程真實截圖佐證 :

1. 兩個物件都用 **ADT** 讀回 `version=active` 確認內容跟寫入的一致 (已完成 , 見 Lecture 程式碼區塊)
2. 標準物件 `C_SlsDocFlfllmntAnalyzer` 原始碼在擴充生效前後完全一致 (已用 ADT 讀回兩次比對確認 , 證明「疊加不修改」)
3. **Fiori Launchpad** 實際畫面 : Track Sales Orders 的篩選列多出 `Personnel Number` 欄位 , 跟擴充前的畫面截圖比對確認是全新出現的 (已截圖確認)

思考題

1. 這一課選錯目標物件的過程裡 , `C_SalesOrderTP` 明明也有 `@Metadata.allowExtensions: true` , 也能正常擴充成功——如果當初沒有進一步查它背後的 OData Service 類型 , 會不會誤以為「擴充失敗」是 CDS View Extension 機制本身的問題 ? 這對你以後排查「明明啟用成功但畫面沒反應」這類問題 , 有什麼啟發 ?
2. `SCFD_REGISTRY` 是 SAP GUI 專屬工具 , 在沒有 SAP GUI 的環境 (例如 fe06 用的 BTP ABAP Environment Trial) 要怎麼查證同樣的資訊 ? (提示 : 回頭比較 fe06 的 `Released Objects` 樹狀結構) 兩種查詢方式 , 一個是「先找 App 再找 View」 , 另一個是「先看整個系統開放了哪些物件」 , 你覺得各自適合什麼情境 ?
3. 這一課的兩個 CDS View Extension (正確與錯誤的版本) 都建立在同一個標準 View 家族 (都是 Sales Order 相關) , 如果之後有第三個人也想擴充 `C_SlsDocFlfllmntAnalyzer` 加另一個不同的欄位 , 會不會跟這一課的 `ZC_SOFANALYZER_VE01` 衝突 ? (提示 : 官方文件提過「一個 View 可以有一個以上的 View Extension」)

答案

見 `ZC_SOFANALYZER_VE01` (DDLs) 、 `ZC_SOFANALYZER_VE01_MDE` (DDLX) , 皆於 on-prem 系統 S4H/130 的 \$TMP 套件。錯誤示範保留在 `ZC_SALESORDER_VE01 / ZC_SALESORDER_VE01_MDE`。沒有修改任何 SAP 標準物件——`C_SalesOrderTP / C_SlsDocFlfllmntAnalyzer` 的原始碼全程未變 , 已用 ADT 讀回驗證 ; Fiori Launchpad 端對端驗證成功 , `Personnel Number` 篩選欄位已確認出現在真實畫面上。